UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA

FACULTAD DE INGENIERIA INFORMATICA Y SISTEMAS

ESCUELA PROFESIONAL DE INGENIERIA EN CIBERSEGURIDAD

PROYECTO WEB CON HTML, CSS Y JAVASCRIPT:

SIMULADOR DE PHISHING

Alumnos : Rivera Panduro, Cristian Domingo

Travesaño Rodríguez, Lizandro Ramón

Docente : Ing. Jorge Luis Jara Linares

Curso : Programación 2

GitHub : https://github.com/mciber3/simulador-phishing-equipo6.git

Ciclo : 2026-II

Fecha : 11/11/26

Tingo María – Perú

2026

I. RESUMEN E INTRODUCCIÓN

El presente informe técnico documenta el diseño, construcción, validación y despliegue del Simulador de Phishing, desarrollado en el curso de Programación II de la Escuela Profesional de Ingeniería en Ciberseguridad de la Universidad Nacional Agraria de la Selva (UNAS). La aplicación se construyó utilizando exclusivamente estándares nativos (HTML5, CSS3 y JavaScript puro), sin frameworks, librerías externas ni CDN. Su arquitectura integra tres páginas semánticas (index.html, crear.html y resultados.html), persistencia local con localStorage, aleatorización mediante el algoritmo Fisher-Yates, gestión CRUD con filtros combinados, exportación en JSON y seis controles defensivos de seguridad.

En ciberseguridad, el factor humano continúa siendo el eslabón más vulnerable frente a campañas de ingeniería social y correos fraudulentos (phishing) que suplantan dominios institucionales o bancarios mediante técnicas como el typosquatting (0ffice365-support.com o un4s.edu.pe) y mensajes de urgencia extrema. Dado que la capacitación teórica suele ser insuficiente para desarrollar reflejos de detección, este proyecto implementa un entorno práctico y seguro en el navegador donde el usuario clasifica correos simulados, recibe retroalimentación inmediata con las pistas que debió notar y consulta su desempeño sin exponerse a enlaces maliciosos reales.

II. OBJETIVOS

2.1. Objetivo General

- Desarrollar un simulador web *responsive* con HTML5, CSS3 y JavaScript nativo para entrenar a los usuarios en la detección de correos de *phishing*.

2.2. Objetivos específicos

- Maquetar una interfaz semántica de tres páginas con enfoque *mobile-first*, variables CSS, Flexbox, Grid y heurísticas de usabilidad de Nielsen.
- Programar la persistencia en localStorage y la lógica del simulador con orden aleatorio *Fisher-Yates*, pistas de retroalimentación, operaciones CRUD, filtros y ranking.
- Implementar controles de seguridad anti-XSS, neutralización de enlaces y validaciones accesibles, verificando el funcionamiento mediante casos de prueba manuales.

III. DISEÑO Y ARQUITECTURA

3.1. Arquitectura modular y flujo funcional

El sistema atiende a dos perfiles: el personal en entrenamiento, que evalúa correos simulados y consulta el ranking, y el capacitador, que administra el banco de casos. El código se organizó separando estrictamente los datos de la vista:

- **Vistas HTML5 (index.html, crear.html, resultados.html):** Estructuradas con etiquetas semánticas (<header>, <nav>, <main>, <section>, <article>, <footer>), atributo lang="es", codificación UTF-8, <title> único por documento y menú común que resalta la vista activa mediante aria-current="page".
- **Capa de datos (js/datos.js):** Inicializa **10 casos simulados** (6 precargados con origen base y 4 creados por el equipo con origen equipo) y encapsula la lectura/escritura en localStorage (equipo6_casos y equipo6_intentos) usando JSON.stringify y JSON.parse dentro de bloques try/catch.
- **Capa de interacción (js/app.js):** Al iniciar una partida en index.html, clona los casos y aplica el algoritmo **Fisher-Yates** para mostrarlos en orden aleatorio sin repetición. Tras elegir «Es Legítimo» o «Es Phishing», revela de inmediato si hubo acierto junto con 1 a 3 pistas de detección, y al finalizar muestra el puntaje, los correos fallados y guarda el intento. En crear.html gestiona el CRUD completo (crear, listar, editar y eliminar con confirmación), búsqueda por asunto, dos filtros (tipo y origen), ordenamiento (A-Z / Z-A), restauración de ejemplos sin duplicar y exportación a JSON. En resultados.html calcula el total de intentos, el promedio general, el Top 5 mejores puntajes y el historial filtrable por alias y ordenable por porcentaje.
- **Flujo en GitHub:** El desarrollo avanzó en 5 ramas (etapa-1-estructura a etapa-5-entrega) integradas a main mediante *Pull Requests*, *commits* convencionales (feat:, fix:, docs:) y etiquetas v0.1 a v1.0, desplegándose en **GitHub Pages**

3.2. Diseño responsive y usabilidad

La hoja css/estilos.css aplica enfoque mobile-first con box-sizing: border-box y variables en :root (--principal: #16325c, --secundario: #2a528a, --fondo: #f2f5f8, --error: #b91c1c, --exito: #15803d) que cumplen el contraste AA de WCAG 2.2. La interfaz responde a tres anchos de pantalla: en móvil (hasta 600 px) el menú se pliega en el botón #menuBoton (aria-expanded) y los filtros van en 1 columna; en tableta (601 a 960 px) el

menú pasa a fila horizontal con Flexbox y los filtros a 2 columnas con CSS Grid; y en escritorio (más de 960 px) los filtros se distribuyen en 4 columnas.

Asimismo, se aplicaron 5 heurísticas de Nielsen y criterios de accesibilidad:

1. Visibilidad del estado: Indicador de progreso ("Caso 1 de 10"), contadores dinámicos y avisos de confirmación.
2. Relación con el mundo real: Formato visual de bandeja de entrada (De:, Asunto:) y botones claros.
3. Control y libertad: Botón «Cancelar edición» en el formulario y diálogo confirm() antes de eliminar registros.
4. Prevención de errores: Deshabilitación de botones (disabled) tras responder cada caso y restricción de longitud y caracteres.
5. Reconocimiento antes que recuerdo: Visualización inmediata de la caja de pistas tras cada respuesta.

Los errores de validación se asocian a cada control mediante aria-describedby y aria-invalid="true" sobre etiquetas <small class="error">, acompañados de foco visible (:focus-visible) para navegación con teclado.

IV. PRUEBAS MANUALES DOCUMENTALES

Se ejecutaron seis casos de prueba manuales para verificar los requisitos funcionales, la prueba de aceptación del Equipo 6 y la resistencia ante entradas inválidas:

Tabla 1. Casos de prueba manuales ejecutados en el Simulador de Phishing

ID	Función evaluada	Pasos y datos de entrada	Resultado esperado	Resultado
CP-01	Retroalimentación de pistas y registro en historial	1. Iniciar intento con alias estudiante_unas. 2. Ante un correo de phishing (seguridad@0ffice365-support.com), pulsar «Es Legítimo». 3. Completar los 10 casos e ir a resultados.html.	Indica respuesta incorrecta, muestra al instante las pistas del caso y al terminar registra el puntaje en el historial y ranking, persistiendo al recargar.	Cumple
CP-02	Orden aleatorio sin repetición	Iniciar dos intentos seguidos y anotar la secuencia de los 10 casos mostrados.	Los 10 casos aparecen en orden distinto en cada intento y ninguno se repite dentro del mismo intento.	Cumple
CP-03	Validación de campos y bloqueo anti-XSS	En crear.html, intentar guardar con remitente invalido, asunto <script>alert(1)</script> y sin pistas.	Bloquea el envío, activa aria-invalid="true", muestra mensajes junto a cada campo y rechaza < y >.	Cumple
CP-04	Enlaces desactivados en correos	En el simulador, hacer clic sobre la URL azul subrayada [https://login.0ffice365-support.com/auth/login].	La URL se visualiza azul y subrayada, pero es un de texto plano y no abre ni redirige a ninguna página.	Cumple

CP-05	CRUD, filtros, restauración y JSON	1. Filtrar por Phishing y Equipo. 2. Editar un asunto y guardar. 3. Eliminar un caso base con confirmación, pulsar «Cargar los 10 casos de ejemplo» y «Exportar datos en JSON».	Filtra al instante, la edición persiste tras F5, restaura el caso faltante sin duplicar los demás y descarga el archivo .json.	Cumple
CP-06	Diseño responsive en 3 anchos	Redimensionar el navegador a 375 px (móvil), 768 px (tableta) y 1200 px (escritorio).	En móvil el menú usa el botón con aria-expanded; en tableta los filtros pasan a 2 columnas y en escritorio a 4 columnas.	Cumple

Fuente: Elaboración Propia

V. SEGURIDAD IMPLEMENTADA

Se verificaron los seis controles de seguridad exigidos:

1. **Datos ficticios y sin secretos:** El código y el historial de Git contienen únicamente casos educativos simulados y alias ficticios, sin credenciales ni claves reales.
2. **Prevención de XSS con textContent y sin eval():** Toda inserción de datos del usuario en el DOM se realiza mediante crearEtiqueta() empleando nodo.textContent = texto y document.createTextNode(), evitando innerHTML con entradas de usuario y eval().
3. **Validación en cliente con JavaScript:** Se validan formatos de correo y alias con expresiones regulares, longitudes mínimas/máximas, cantidad de pistas (1 a 3) y bloqueo de símbolos < y >, considerando que en producción toda validación cliente debe replicarse en el servidor.
4. **Neutralización de enlaces en correos:** La función mostrarCuerpoCorreo() detecta cadenas que inician con http:// o https:// y las convierte en nodos con textContent, imitando la apariencia azul subrayada de un enlace sin crear etiquetas <a> activas.
5. **Uso seguro de localStorage:** Al carecer de cifrado y tener un límite aproximado de 5 MB por dominio expuesto a lecturas por scripts, solo se almacenan las listas públicas equipo6_casos y equipo6_intentos, nunca contraseñas ni datos sensibles.
6. **Política de Seguridad de Contenido (<meta> CSP):** Los tres documentos HTML integran <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; style-src 'self'; object-src 'none'; base-uri 'none'; form-action 'self'">, impidiendo scripts en línea y recursos de terceros.

VI. REFLEXION

6.1. Reorganización con un framework front-end

Al construir este simulador desde cero con JavaScript puro, comprendimos cómo funciona realmente el árbol del DOM por dentro, pero también notamos lo trabajoso que

resulta mantener la interfaz sincronizada cuando el proyecto crece. Por ejemplo, tuvimos que copiar el mismo bloque <header> del menú en los tres archivos HTML y programar funciones en js/app.js que borran (innerHTML = "") y vuelven a crear todas las filas <tr> de las tablas cada vez que el usuario escribe una letra en el buscador. Si reorganizáramos nuestro proyecto utilizando un framework como React o Vue, pasaríamos de manipular nodos manualmente a trabajar con una arquitectura declarativa de componentes reutilizables y estado reactivo:

- <BarraNavegacion/>: Unificaría el encabezado y el estado abierto (aria-expanded) del menú móvil en un solo archivo para las tres vistas.
- <TarjetaCorreo caso="{casoActual}"/>: Recibiría por propiedades (props) los datos del correo y transformaría las URLs en etiquetas automáticamente, aprovechando además la protección anti-XSS nativa del framework.
- <CampoFormulario/> y <TablaDatos/>: Reutilizarían la estructura de validaciones (aria-invalid) y actualizarían solo las filas filtradas en memoria sin reconstruir toda la tabla en cada búsqueda.

Finalmente, reemplazaríamos la recarga de tres páginas .html por un enrutador de página única (SPA) y sincronizaríamos localStorage de forma reactiva, dejándolo listo para conectarse en el futuro a una base de datos real con validación en servidor.

VII. CONCLUSIONES

- Se desarrolló el Simulador de Phishing del Equipo 6 con HTML5, CSS3 y JavaScript nativo, logrando un entorno web funcional de tres páginas para entrenar en la detección de correos fraudulentos.
- Se maquetó una interfaz semántica *mobile-first* con variables CSS, Flexbox y Grid adaptable a tres anchos de pantalla (móvil, tableta y escritorio), integrando cinco heurísticas de usabilidad de Nielsen.
- Se programó la persistencia en localStorage (js/datos.js) y la lógica interactiva (js/app.js) con orden aleatorio *Fisher-Yates*, pistas inmediatas, operaciones CRUD, filtros combinados, ranking Top 5 y exportación JSON.
- Se implementaron y validaron mediante seis pruebas manuales los controles de seguridad y accesibilidad, previniendo ataques XSS y redirecciones reales mediante textContent, Content-Security-Policy, aria-invalid y enlaces desactivados.